

multiple question 25/4/26
MGT

Multi-Threaded Programming

✓ Build-in support

✓ Description: Contains two or more parts that can run concurrently. Each part of such a program is called a thread.

↳ Each thread defines a separate part of execution.

"Multithreading is a specialized form of multitasking."

Two types of multi-tasking

Process-based Multitasking
(Multiprocessing)

Thread-based Multitasking
(Multithreading)

✓ Switching is costly
(Cost of communication between the process is high)

Multithreading

✓ Each thread can function independently

✓ Thread is a light weight sub-process of process

✓ Shares same memory address

Thread States

Shayan

- ① New
- ② Runnable
- ③ Running
- ④ Waiting
↳ Timeout
- ⑤ Terminated

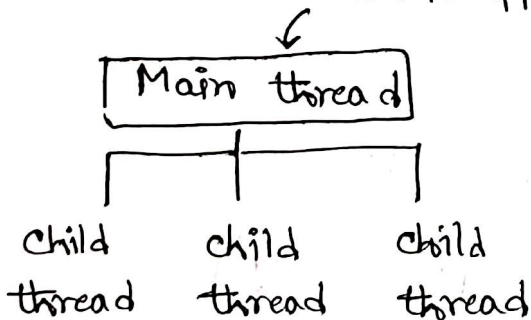
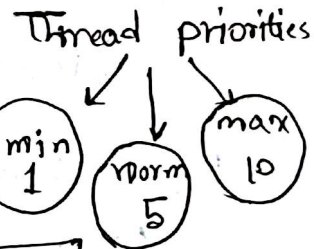
Figure of Thread Life Cycle

Explain thread life cycle

{ Main method
≈ Main thread }

States of thread states :

- ① getName : Get running thread's name.
- ② getPriority : Get thread's priority
- ③ isAlive : Determine if a thread is still running.
- ④ join : Wait for a thread to terminate.
- ⑤ run : Entry point for the thread.
- ⑥ sleep : Suspend a thread for a period of time (ms)
- ⑦ start : Start a thread by calling its run method.



currentThread()

↑ get running thread
↓ currently running
depression()

<main thread>

Shayan

```
class CurrentThreadDemo {
```

```
    public static void
```

```
        Thread t = Thread.currentThread();
```

```
        System.out.println("Current thread: " + t);
```

```
        t.setName("My Thread");
```

```
        System.out.println("After name change: " + t);
```

```
        try {
```

```
            for (int n = 5; n > 0; n--) {
```

```
                System.out.println(n);
```

```
                Thread.sleep(1000);
```

```
            }
```

```
        }
```

```
        catch (InterruptedException e) {
```

```
            System.out.println("Thread interrupted");
```

```
        }
```

```
    }
```

```
}
```

Creating a Thread

Two ways—

① by extending Thread class,

② by implementing the Runnable interface.

Thread Constructors

Shoyan

- ① Thread
- ② Thread (String name)
- ③ Thread (Runnable r)
- ④ Thread (Runnable r, String name)

#Creating Thread

① Extending Thread

```
class Multi extends Thread {  
    public void run () {  
        System.out.println("Thread running");  
    }  
}
```



```
public static void main (String args[]) {  
    Multi m1 = new Multi();  
    m1.start();  
}
```

```
}  
class th extends Thread {  
    public void main run () {  
        for (int i=0; i < 5; i++) {  
            System.out.println(i);  
        }  
        try { Thread.sleep(100); } catch (e) { sout(e); }  
    }  
}
```

```
public class Main {  
    psvm(...) { th t1 = new th();  
        th t1.start(); } }
```

④

Thread: join

Shayan

```
th t1 = new th();  
th t2 = new th();  
t1.start();  
try { t1.start();  
    t1.join();  
} catch (Exception e) { ... }  
t2.start();
```

Creating a thread

② Implementing the Runnable interface

```
class M implements Runnable {  
    public void run() {  
        System.out.println("Thread is running.");  
    }  
}
```

```
public class Main {  
    public static void main(...) {  
        M m1 = new M();  
        Thread t1 = new Thread(m1);  
        t1.start();  
    }  
}
```

~~class th extends Thread {~~

class th implements Runnable {

public void run() {

for (int i=0; i<5; i++) {

System.out.println("i: " + i);

}

try {

Thread.sleep(500);

} catch (InterruptedException e) {

System.out.println(e);

}

}

}

public class Main {

psvm(...) {

th m1 = new th();

Thread t1 = new Thread(m1);

th m2 = new th();

Thread t2 = new Thread(m2);

t1.start();

try { t1.join(); } catch (Exception e) { }

t2.start();

}

}

```
class oddThread extends Thread {
```

```
    public void run() {
```

```
        for (int i = 1; i < 10; i += 2) {
```

```
            System.out.println("i: " + i);
```

```
        }
```

```
        try { Thread.sleep(100); }
```

```
        catch (InterruptedException e) {
```

```
            System.out.println(e);
```

```
        }
```

```
    }
```

```
}
```

```
class evenThread implements Runnable {
```

```
    public void run() {
```

```
        for (int i = 0; i < 10; i += 2) {
```

```
            System.out.println("i: " + i);
```

```
        }
```

```
        try { Thread.sleep(100); }
```

```
        catch (InterruptedException e) {
```

```
            System.out.println(e);
```

```
        }
```

```
    }
```

```
}
```

```
public class Main {
    psvm(...) {
```

```
        OddThread ot = new OddThread();
        ot.start();
```

```
        try { ot.join() } catch (Exception e) {
            System.out.println(e); }
    }
```

```
        evenThread et = new evenThread();
```

```
        evenThread et-wrap =
```

```
        Thread et-wrap = new Thread(et);
```

```
        et-wrap.start();
```

```
    }
```

```
}
```